

TopImmo

Mietwohnungen finden.
Nach Stadtbezirk auswählen.

Angular · C# / ASP.NET Core · SQLite
Immobilien­daten über RapidAPI



Eine Karte führt zum passenden Angebot.

Ort auswählen, geladene Angebote eingrenzen, Details ansehen.

01 Anmelden

Zugang zur Übersicht

02 Bezirk wählen

Karte oder Legende

03 Angebot öffnen

Preis, Fläche und Bilder



Aktuelle Demoaufnahme · 04.10.2026

28 Immobilien gefunden.



Kompaktes Wohlfühlomizil -
Wohnungen in ruhiger Lage i...

875 €

28 m², 1 Zimmer

Bussenstraße 37, 70184 Stuttgart, Ost

Zum Angebot



Home & Co – Easy Living |
Möbliertes All-Inclusive...

709 €

13 m², 1 Zimmer

Nauheimer Straße 78-80, 70372
Stuttgart, Bad Cannstatt

Zum Angebot



Vier technische Pflichtbestandteile.

Frontend, Backend, externe Datenquelle und Datenbank bilden den Produktweg.

Angular-Frontend

Formulare und Navigation
Karte, Angebotsliste und Details

C#-Backend

ASP.NET Core / .NET 9
HTTP-Endpunkte, Verarbeitung und Zugriffsschutz

Externe API

Immobilienuche über RapidAPI
JSON als Datenaustausch

SQLite

Benutzer und Refresh-Tokens
Speicherung über Entity Framework Core

Zwei Datenwege im selben Backend.

Die Oberfläche nutzt das eigene Backend über HTTP und JSON.

Angular · Formulare, Karte, Liste, Details

↓ HTTP / JSON + Bearer-Token

ASP.NET Core · Controller und Services

Authentifizierung

↓ EF Core → SQLite
Benutzer + Refresh-Tokens

Immobilien

↓ RapidAPI → Verarbeitung
Mietangebote + 5-Minuten-RAM-Cache

Aus JSON werden Bezirksdaten.

Die Trefferzahlen beschreiben nur die geladenen Mietangebote.

- 1 Laden** Cache nutzen oder externe API abfragen
- 2 Filtern** Nur Mietwohnungen: apartmentrent
- 3 Zuordnen** Ortsnamen → Stadtteil → Stadtbezirk
- 4 Zählen** Treffer je Bezirk gruppieren und sortieren
- 5 Auswählen** Angebotsliste für den gewählten Bezirk liefern

5 Minuten

RAM-Cache
Ladesperre gegen parallele
Doppelabrufe

API: Seite 1, maximal 30 Treffer.
Keine vollständige
Marktstatistik.

SQLite speichert Konten und Sitzungsdaten.

Ein Benutzer kann mehrere Refresh-Tokens besitzen.

Users

Id · Primärschlüssel
Email
PasswordHash · BCrypt

1 → n

RefreshTokens

Id · Primärschlüssel
Token · ExpiresAt · IsRevoked
UserId · Fremdschlüssel

Immobilienangebote liegen im RAM-Cache; sie werden derzeit nicht in SQLite gespeichert.

JSON, SQLite und der 5-Minuten-Cache.

C# verarbeitet die Daten; RAM-Cache und SQLite haben unterschiedliche Aufgaben.

Immobilien Daten: JSON → RAM

HttpClient → JSON → ImmoScoutResponse
List<Listing> → Mietfilter → Bezirkszuordnung
IMemoryCache → JSON-Antwort an Angular

Anmeldedaten: JSON → SQLite

RegisterRequest → BCrypt-Hash → User
AppDbContext → SaveChangesAsync()
SQLite-Tabellen: Users und RefreshTokens

5 Minuten IMemoryCache + SemaphoreSlim(1,1)

Absoluter Ablauf ab Speichern; Lesen verlängert nicht.
Erst die nächste Anfrage nach Ablauf lädt neue Daten.
Cache gilt pro Prozess; Neustart leert den RAM.

Cache prüfen → WaitAsync() → nochmals prüfen.
Eine Anfrage lädt; die anderen nutzen danach den Cache.
finally: Release() gibt die Sperre auch bei Fehler frei.

Der Server entscheidet über den Zugriff.

Frontend und Backend haben unterschiedliche Aufgaben.

Passwort prüfen

BCrypt-Hash
Registrierung und Login
über Auth-Endpunkte

JWT verwenden

Bearer-Token in Anfragen
Backend prüft Gültigkeit
[Authorize] schützt Angebote

Token erneuern

Refresh-Endpoint
rotiert Refresh-Tokens
Bereits genutzte werden
abgewiesen

Nächster Schritt: automatischen Refresh und Sitzungsauslauf im Browser zuverlässig behandeln.

Frontend sendet Tokens; das Backend prüft sie.

Login liefert einen signierten Access-Token und einen zufälligen Refresh-Token.

JWT = Header . Payload . Signatur | HMAC-SHA256 (HS256)

Payload: Benutzer-ID, E-Mail, iss, exp. Inhalt ist lesbar; die Signatur schützt vor unbemerkten Änderungen.

Angular / Browser

localStorage: Access + Refresh
Guard prüft: Token vorhanden?
Interceptor setzt Bearer <JWT>.
Die Gültigkeit prüft der Server.

ASP.NET Core / Server

Prüft Signatur, Aussteller und Ablauf.
[Authorize] schützt Immobilien-API.
User-ID: Namelidentifier; kein sid.
Audience-Prüfung derzeit aus.

Refresh / SQLite

64 Zufallsbytes → Base64; 7 Tage
Token + UserId + Ablauf + Widerruf
Refresh: alten Token widerrufen,
neuen Refresh + Access ausgeben.
Access: Standard 60 Minuten

Noch offen: automatischer Refresh / 401-Behandlung. Logout löscht Tokens im Browser, ohne Serverwiderruf.

Vom Login bis zur Detailansicht.

Zwei Minuten: einen verständlichen Ablauf zeigen.

01

Login

02

Bezirk wählen

03

Angebot öffnen

04

Bild wechseln

05

Zurück + Logout



28 Immobilien gefunden.

**Kompaktes Wohlfühlomizil -
Wohnungen in ruhiger Lage i...****875 €**

28 m², 1 Zimmer

Busenstraße 37, 70184 Stuttgart, Ost

[Zum Angebot](#)**Home & Co – Easy Living |
Möbliertes All-Inclusive...****709 €**

13 m², 1 Zimmer

Nauheimer Straße 78-80, 70372
Stuttgart, Bad Cannstatt[Zum Angebot](#)**Kompaktes Wohlfühlomizil - Wohnungen in ruhiger Lage in Stuttgart-Ost,
zwischen 690 und 730 Euro****875 €**28 m², 1 Zimmer
Busenstraße 37, 70184 Stuttgart, Ost
Stadtbezirk: Ost
vor 9 Tagen

875 €

28 m²

1 Zi.



Lokale Demo · gültiger API-Zugang erforderlich

Gesicherte Aufnahmen vom 04.10.2026

Welche Technologien braucht der lokale Start?

Die Versionen folgen dem vorhandenen Projekt; SQLite benötigt keinen separaten Server.

Frontend

Node.js 24.x + npm
Angular / CLI 21.2.x
TypeScript 5.9.x + RxJS

Backend

C# · .NET 9 SDK · Ziel net9.0
ASP.NET Core + EF Core 9
SQLite-Provider + BCrypt

Lokal vorhanden

Node 24.21.0 · npm 11.19.0
.NET 9 SDK: 9.0.318
.NET / ASP.NET Runtime: 9.0.20

1 · Konfiguration

node scripts/setup-local-config.mjs

2 · Backend :5197

cd Backend/ImmscoutAPI → dotnet run --launch-profile http

3 · Frontend :4200

cd Frontend/TomInnoFrondEnd → npm ci → npm start

Browser: <http://localhost:4200> · JWT-Key lokal erzeugen; eigener RapidAPI-Key + Internet für Liveangebote.